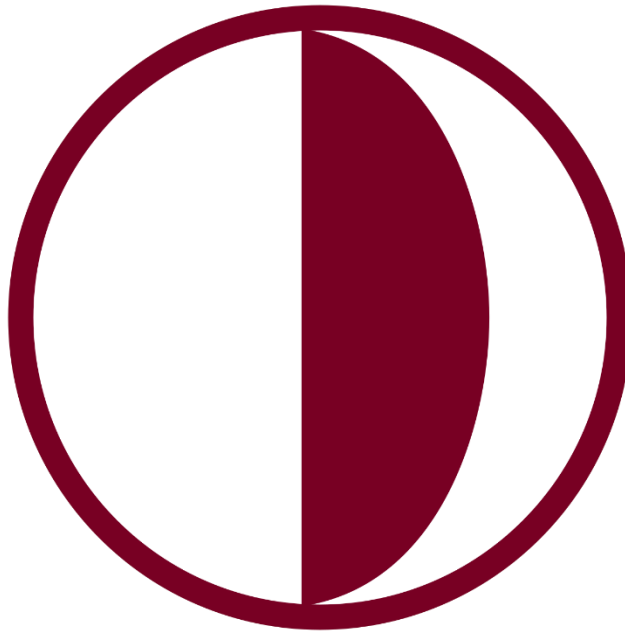




NEAR EAST UNIVERSITY

FACULTY OF ARTIFICIAL INTELLIGENCE AND INFORMATICS

DEPARTMENT OF SOFTWARE ENGINEERING

KIDS LANGUAGE LEARNING SYSTEM

Final Year Project

SWE - 491

Submitted by: Cindy Katua and Mary Chisoanyi

Student IDs: 20233406; 20226411

Supervisor: Clifford Peter Mlimwa

Academic Year 2025 – 202

TABLE OF CONTENTS

Contents

TABLE OF CONTENTS	2
ACKNOWLEDGEMENT	3
ABSTRACT	4
LIST OF ABBREVIATIONS.....	5
INTRODUCTION	5
1.1 Project Overview	6
1.2 Problem Statement	6
1.3 Aim of the Project	6
1.4 Project Scope and Use Cases	6
1.5 Why This Project Matters	7
LITERATURE REVIEW.....	7
2.1 Educational Technology Overview	7
2.2 Existing Language Learning Platforms	7
2.3 Limitations of Current Systems	8
2.4 Summary	8
SYSTEM ANALYSIS	8
3.1 Stakeholder Identification.....	8
3.2 Functional Requirements	9
3.3 Non-Functional Requirements	10
3.4 Use Case Analysis	10
SYSTEM DESIGN	11
4.1 Overall System Architecture	11
Presentation Layer (Frontend)	11
Application Layer (Backend).....	11
Data Layer (MongoDB Atlas).....	11
4.2 Use Case Diagram	12
4.3 Activity Diagram.....	Error! Bookmark not defined.
4.4 Component Diagram	13
4.5 Entity Relationship Diagram.....	13
SYSTEM IMPLEMENTATION	14
5.1 Technologies Used	14
5.2 Frontend Implementation.....	14
General Design Principles	14
Login and Registration	Error! Bookmark not defined.
Main Menu.....	15

Lesson Module	16
Quiz Module	17
Games Module	19
5.3 Backend Implementation	20
Database Setup	20
Implemented Modules	21
API Architecture	22
5.4 Security Considerations	23
TESTING AND VALIDATION	25
6.1 Testing Strategy.....	25
6.2 Test Cases	25
6.3 Test Results and Evaluation.....	27
Route Configuration Errors	Error! Bookmark not defined.
MongoDB Integration Issues	Error! Bookmark not defined.
Frontend-Backend Communication	Error! Bookmark not defined.
RESULT AND DISCUSSION	28
FUTURE WORK AND ENHANCEMENTS	30
CONCLUSION.....	31
REFERENCES	32

ACKNOWLEDGEMENT

We would like to express our gratitude to Near East University, especially regarding its access to knowledge and resources that have been applied for our project. Many important lessons learned, many projects and project work assignments, and many hours of late-night troubleshooting over the years have played a part in making the developer we are today; an easy consequence of each one, and this project is an evident consequence of that. We would like to thank our supervisors as well sincerely because they were patient and very constructive in helping with our project. There were even times, though, when we didn't know how to go forward, and the support that we had made all the difference. Building this project has taught us far more than we expected. We have learnt the importance of teamwork, communication and the ability to function under immense pressure. And finally, to our family and friends, who encouraged us through every step of this journey, your support went a long way.

ABSTRACT

Kids Language Learning System is an interactive educational application for children. It offers lessons, quizzes, games and stories that can help a child strengthen their basic English knowledge. The system was implemented using HTML, CSS and JavaScript for the frontend, and Express.js, Node.js and MongoDB for the backend. The front-end part of the application prioritizes child-friendly interfaces, voice guidance and gamification features to make sure that the application captures the child's attention. The application was developed as per the SDLC, right from the planning phase to the last phase, which was the testing and debugging phase, that was accompanied with iterations of feature refinements. Git and GitHub were used for version control amongst team members to ensure a smooth process. However, the team still faced a couple of problems during the development phase such as merge conflicts during push and pull, failed communication between the frontend and the backend as well as UI response problems. These problems were later solved after proper communication and collaboration between the team as well as immense iterative testing. The system is up and running, with a few pending future enhancements such as making it multilingual and adding administrative dashboards.

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
ERD	Entity Relationship Diagram
GUI	Graphical User Interface
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JS	JavaScript
JSON	Script Object Notation
MVC	Model View Controller
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SRS	Software Requirement Specification

INTRODUCTION

1.1Project Overview

Since the pandemic that occurred in 2019 – 2020, technology has hugely influenced the education sector [1], [2]. Rather than reading flashcards and participating in rote drills, children are now interacting with adaptive, responsive systems that respond appropriately to their current performance [1]. We realized there was an opportunity to enhance the field and create the Kids Language Learning System. A web app designed to support children to understand language learning in numerous ways. It has lessons, quizzes, stories and games. It has vocabulary content that includes colors, animals, alphabets, fruits and shapes. We divided the project implementation into two, whereby one person developed the frontend, and the other person developed the backend. Working hand in hand with GitHub we made this project happen

1.2 Problem Statement

Children these days grow up with their faces glued to screens. Traditional methods of learning are becoming outdated and irrelevant due to their dull, static and non-interactive characteristics [3]. The education sector should take advantage of this through gamified educational systems [4]. However, most of the language applications developed either target older audiences, have priced subscriptions that limit access or just present learning material in a more textual way that may not be suitable for children. These are the gaps that our system will fill.

1.3 Aim of the Project

This project was developed with different aims in mind that may include;

- To build an interactive and child friendly learning system, that will be easily usable.
- To build a system that will use voice guided instructions and gamification features to increase learning engagement.
- To build a system that is not limited by space and time, it will be easily accessible anywhere and at anytime
- To build a system that will help maximize children's early development window. This will allow them to improve their multitasking skills, their ability to focus and remember patterns.
- To build a system that will naturally teach children without them feeling they are being forced to memorize new concepts.
- To build a system that will encourage multilingualism from a young age
- To build a system that will help build a child's confidence and concentration that can later translate into other higher performance subjects

1.4 Project Scope and Use Cases

The main objective of the system is to help build a solid foundation for children to develop their communication skills as well as learn how to properly express themselves in verbal and non-verbal ways. This is achieved using core use cases that include:

- Registration and logging into the system.
- Selecting a language to learn.
- Taking lessons on selected topics such as colors, fruits, shapes, alphabets and animals.
- Playing interactive games that reinforce vocabulary learnt during lessons.
- Reading voice guided stories that help master proper word pronunciation.
- Taking quizzes to ensure lesson understanding and mastery.

1.5 Why This Project Matters

Through this project, children can universally receive free and high-quality education that is easily accessible from anywhere. It also gives children the ability to learn at their own pace, revisit hard concepts and speed ahead in concepts they easily grasp. On the other hand, this project has allowed us to practice most if not all, of the concepts that we have been taught in our four years of university. We can proudly say that we are fluent in software requirement analysis, system architecture and design, software testing and implementing different programming languages.

LITERATURE REVIEW

2.1 Educational Technology Overview

In the recent past, educational technology has grown more rampant in modern education systems [1], [7]. Learning institutions are moving away from static methods of teaching and moving towards technological ways, since they have proven to motivate and retain more learners, particularly the younger ones. Voice guidance and gamification are well-known factors in early-childhood learning tools [3], [5], that bridge the gap between instruction and understanding. For children who have not yet read independently, this is bridged by a medium called audio support. In this system, the introduction of the Browser Speech Synthesis API reflects this wisdom.

2.2 Existing Language Learning Platforms

Before work commenced the team researched an array of language learning platforms in an attempt to grasp what was already offered, what worked well, and where there were missing links. Duolingo. Duolingo is arguably the most popular language learning app in the world. It employs a gamification, bite-sized lesson system with streaks, XP points and a leaderboard to drive daily engagement. But Duolingo does introduce a kids' version (Duolingo ABC), but its official product is targeted at older ones with older users. It was the gamification model it uses which influenced the design of how this system's badge and reward system is constructed [8]. Khan Academy Kids. Khan Academy Kids is a free app for kids ages 2 to 8 that tracks early reading, language and math's. It has animated characters, read-aloud capabilities, and variable difficulty. Key elements of the Khan Academy Kids interface design philosophy were very useful for the guide to develop this system's child friendly interface with large interactive elements with clear audio cues and minimal text. Lingokids. Lingokids is a subscription-based English-learning platform for 2 to 8-year-olds. There an integration of music, interactive games and animated stories to teach vocabulary and fundamental grammar [9]. Lingokids' multi-modal system (audio, visual and game) were the reasons used to develop lessons, quizzes, stories and games separately from each other and together, within the system [10]. Starfall. Starfall is a free phonics site that focuses on learning early reading skills and has served children since 2002. Its interface may feel dated for now, but the sheer longevity of it points to the fact that free good educational tools with good content could still hold water. The choice of browser-based and a free one is based on the same philosophy of what we do here [11].

2.3 Limitations of Current Systems

Even though the existing platforms are helpful, the review presented common problems:

- **Cost and access:** A lot of the best products have subscription paywalls that prevent access to family homes in poorer areas.
- **Personalization limited:** The majority of platforms provide limited content paths that are rigid as well as not scalable to new educational needs or localized languages
- **Excessive dependence on mobile:** Most popular solutions are mobile-only or mobile-first and are unable to be used in classroom or home-computer settings.
- **Few administrator tools:** The majority of consumer-facing platforms lack an effective way for a teacher or parent to manage or check the content.
- **Language is very limited:** Most platforms are more focused on English as a second language for non-native speakers and this leaves a niche for services to support more than one target language equally.

These constraints have influenced our design choices for this project. The system is web-based (not mobile-only), open access, designed with an admin backend in mind, and designed to run across multiple languages through its database structure.

2.4 Summary

Literature review has confirmed both the value of what this project is attempting and the genuine gap it fills. Though robust platforms exist, none provide the exact combination of free access, web-based delivery, multi-language extensibility, admin content management, and a fully open architecture that this system provides. On top of this, the examination confirmed key design decisions taken during the development process—gamification, voice guidance, multi-modal content delivery, modular system architecture separating lessons, quizzes, stories, and games into independently manageable components.

SYSTEM ANALYSIS

Before we could start the development of the project, the team heavily analyzed the system. We identified system stakeholders, boundaries and limits. The whole process helped us acquire deliverables such software requirement specification documents and architectural diagrams that aided in the implementation process.

3.1 Stakeholder Identification

During the system analysis process, we were able to identify three major stakeholders:

The Child User: Children are the primary end users of the system. They directly interact with the system by playing games, taking lessons, learning through stories and taking quizzes.

Administrator: They are responsible for managing the system. They can add lessons, stories, quizzes and increase difficulty of the games as the learners improve.

Parent or Guardian. They are secondary actors in the system whereby they do not directly interact with the system, although they are facilitators of their child's learning. A parent dashboard for progress and activity tracking can be an interesting future enhancement

3.2 Functional Requirements

During the requirement analysis stage, we successfully identified the following functional requirements that were suitable for the system:

FR1 – Authentication

- The system shall allow first time users to register using their names, emails and passwords
- The system shall allow registered users to log in securely
- The system shall store user credentials and session data and manage it through the backend database

FR2 – Language Selection

- The system shall display available languages.
- The system shall display relevant activities of the chosen language.

FR3 – Lesson Module

- The system shall retrieve and display lessons from the database.
- Lessons shall provide vocabulary lessons on colors, animals, alphabets, fruits and shapes

FR4 – Quiz Module

- The system shall provide quizzes for each lesson category
- Quizzes shall be loaded dynamically from the database
- The system shall calculate and display scores upon quiz completion

FR5 – Story Module

- The system shall provide a library of stories accessible through the main menu
- Stories shall be loaded dynamically from the backend
- Voice narration shall be available during story reading

FR6 – Games Module

- The system shall provide at least one educational game per lesson category
- Games shall be loaded dynamically from the backend
- Each game shall include animated feedback for correct and incorrect responses

FR7 – Progress Tracking

- The system shall record completed lessons, quiz scores, and game sessions
- Progress data shall be stored in the database and retrievable per user

FR8 – Badge and Reward System

- The system shall award badges to users upon reaching defined milestones
- Earned badges shall be displayed on the user's profile page

FR9 – Admin Functions

- Admins shall be able to add quizzes, lessons, stories and games
- Admins shall be able to review the children's progress

3.3 Non-Functional Requirements

- Usability: The system should be easy for children to navigate and handle without any parental supervision.
- Performance: The system should be fast and highly responsive to users.
- Security: The system should store user information securely ensuring privacy.
- Reliability: The system should be able to perform without crashing unexpectedly
- Scalability: The system architecture should be flexible enough to allow future growth.
- Maintainability: The system should be easily testable and maintainable to allow future developments

3.4 Use Case Analysis

The main use cases for the project include:

- The child can create a new user account
- The child can log into an existing user account
- The child can select language from the options available
- The child can choose lesson activity
- The child can take a quiz on the vocabulary subject of their choice
- The child can read a story with voice guidance
- The child can play games to strengthen their knowledge on new vocabulary
- The system analyzes the child's performance and adjusts to difficulty
- The administrator can update new lessons, quizzes, games and stories into the system
- The administrator can monitor and review the child's progress

SYSTEM DESIGN

In this section, we will document the architectural design decisions we have made for our system.

4.1 System Architecture

The system uses three-layer architecture. This architecture allows the system to be separated into different parts that will clearly and efficiently perform their own tasks. These layers include; the Presentation layer, the Application layer and lastly the Data layer

Presentation Layer

This layer consists of the frontend part of the system that is developed using HTML, CSS and JavaScript technologies [13], [14]. It consists of everything the user interacts with directly on the screen, that is, creating new accounts, logging into existing ones, playing games, reading stories, taking lessons and quizzes. This layer communicates with the backend using Fetch API requests that load the contents onto the modules dynamically then displayed to the user.

Application Layer

This layer consists of the backend part of the system that was developed using Node.js and Express.js [15], [16]. It handles the main logic and functionality of the whole system. The backend is organized into different route files such as users, languages, lessons, quizzes, stories, games, badges and progress. Each route processes requests that aid the frontend to efficiently communicate with the database.

Data Layer

This layer consists of the database part of the project. It uses MongoDB Atlas as the cloud database for storing any data within the system [17]. It contains collections for users, languages, lessons, games, quizzes, stories, progress and badges

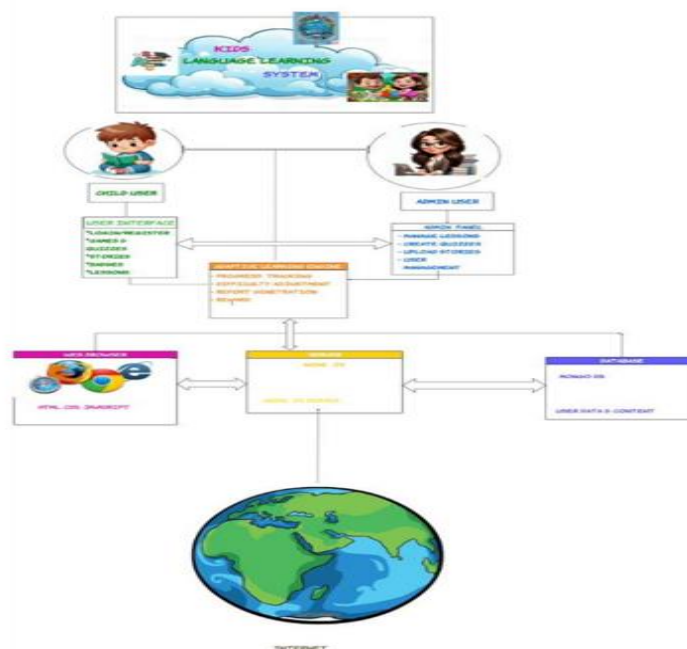


Figure 1: System Architecture

4.2 Use Case Diagram

The use case diagram shows how external entities interact with the system. There are two main actors, the child and the system admin. Corresponding use cases are highlighted in the diagram below.

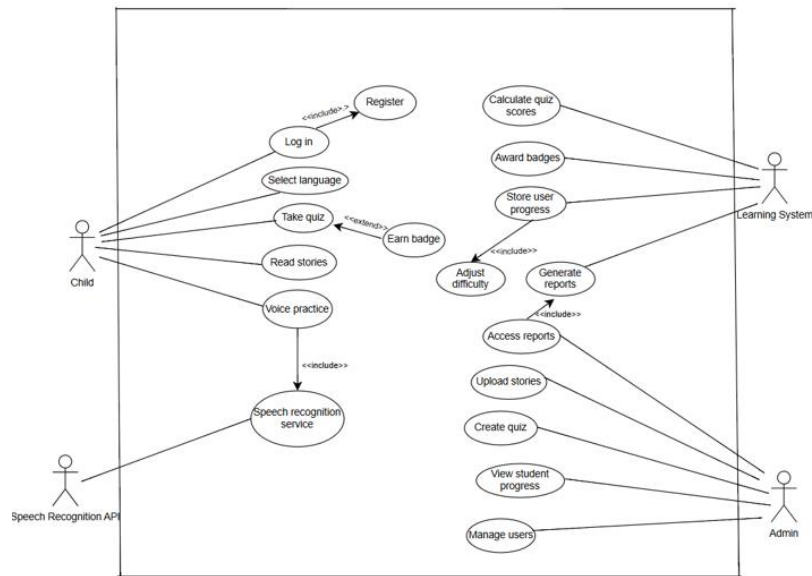


Figure 2: Use Case Diagram

4.3 Activity Diagram

The activity diagram shows the step-by-step flow of events from the child's perspective as they interact with the system. The process begins with the child logging into their account, if they don't have one, they are redirected to register. After successfully logging in, they select the language they want to learn of which English is the only available language currently. After they choose the activity, they want to complete. Based on the child's performance the system decreases or increases difficulty to match their progress. Finally, the system stores the child's progress in the database.



Figure 3: Activity Diagram

4.4 Component Diagram

The component diagram shows the main parts of the system and how they interact with each other. The system has a user interface layer that is used by both the child user and the admin. Through this interface they are able to access different features of the application. The main modules include the authentication module, the lesson module, the quiz module, the story module and the badge module that shows user progress. These modules work hand in hand with the application layer that processes user requests and manages the system functions. The application layer includes two important backend components, progress tracking and badge components.

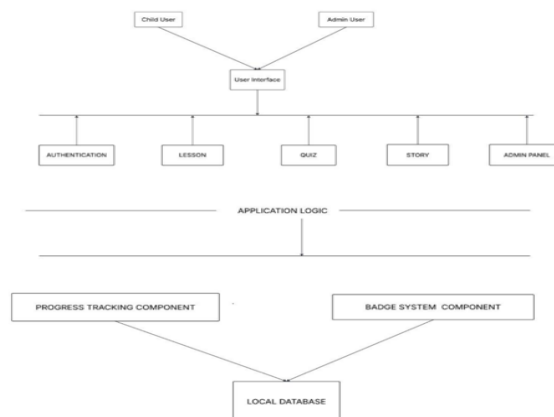


Figure 4: Component Diagram

4.5 Entity Relationship Diagram

The Entity-Relationship Diagram (ERD) below represents the overall relationships among the main entities in the system. There are six essential entities that form the backbone of this system: User, Language, Lesson, Quiz, Progress, and Badge. Multiple records can exist as a user interacts with their coursework in the Progress entity table. Each language can be divided into various lessons, each containing a specific number of quizzes to be completed. The Progress entity has an associated User, Lesson, and Quiz so that their scores and completion rates can be always monitored.

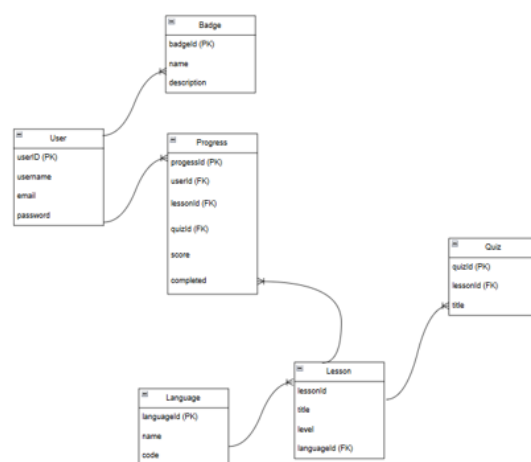


Figure 5: Entity Relationship Diagram

SYSTEM IMPLEMENTATION

5.1 Technologies Used

Technology	Purpose
HTML5	Frontend page structure and layout
CSS3	Styling, responsive design, and animations
JavaScript	Frontend interactivity and game logic
Node.js	Backend runtime environment
Express.js	Backend routing and REST API
MongoDB Atlas	Cloud-hosted database storage
Mongoose	MongoDB object modelling for Node.js
Thunder Client	API endpoint testing
Git & GitHub	Version control and team collaboration
VS Code	Development environment
localStorage API	Client-side session state management
Browser Speech Synthesis API	Voice guidance and pronunciation

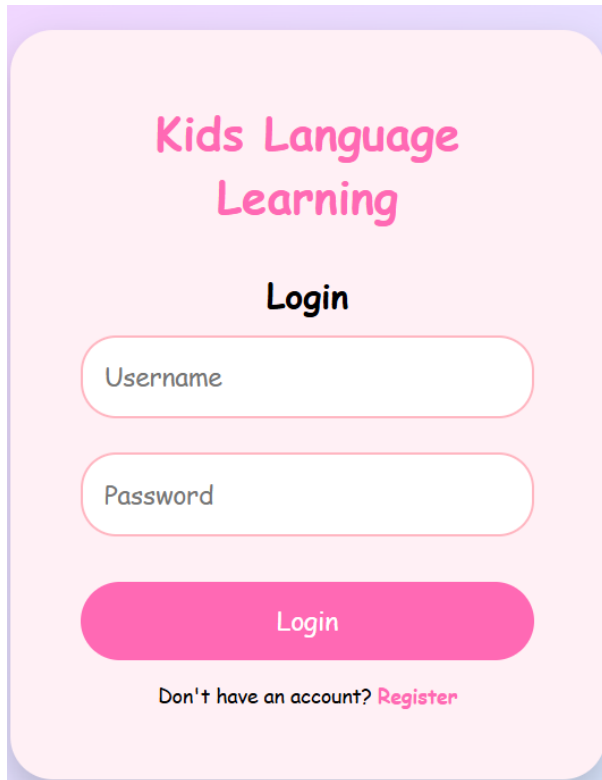
5.2 Frontend Implementation

General Design Principles

We wanted to keep a full knowledge of the target audience while working on the frontend and tried to address only very young children in the initial stages of reading. The intent was to allow these kids time for independent participation. Hence, the design had to focus on simplicity of use i.e. big buttons, bright colors, appealing graphics, and voice navigation leading a child through the experience. We also added animation and immediate feedback that were designed to entertain & help kids to use the interface. The frontend was separated into different HTML, CSS, and JS files, which focused on the different areas for the platform (lessons, quizzes, stories, games, etc.). This framework can really help with keeping things current and can enable changes later. Most of the pages are producing content using backend API instead of the normal hard coded content [18].

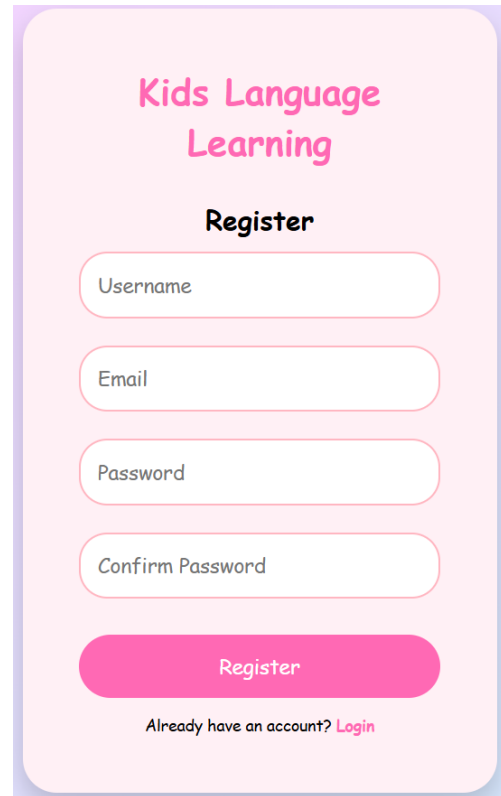
Login and Registration

Additionally the frontend was put in place an authentication system with registration and login functions. This makes it possible for users to create accounts securely and log in again regularly. When users create a new account, they should fill in necessary fields which include usernames, email address and passwords. For security and to prevent the possibility of errors, LocalStorage was present to temporarily store the user session information.



The login page features a light pink background with rounded corners. At the top, the text "Kids Language Learning" is displayed in a bold, pink font. Below this, the word "Login" is centered in a bold, black font. There are two input fields: "Username" and "Password", both with rounded rectangular borders. Below the input fields is a large, rounded pink button labeled "Login". At the bottom, there is a link that says "Don't have an account? Register" in a smaller, pink font.

Figure 6: Login Page



The registration page has a light pink background with rounded corners. At the top, the text "Kids Language Learning" is displayed in a bold, pink font. Below this, the word "Register" is centered in a bold, black font. There are four input fields: "Username", "Email", "Password", and "Confirm Password", all with rounded rectangular borders. Below the input fields is a large, rounded pink button labeled "Register". At the bottom, there is a link that says "Already have an account? Login" in a smaller, pink font.

Figure 7: Registration Page

Main Menu

The moment a user clicks on the main menu, they are welcomed with the friendly design which focuses on lessons, quizzes, games, stories and badges. This makes the process simple and user friendly, whether lessons, quizzes, games, badges, or any other content provided on the site. This changed a lot as the app developed as the main menu's design was heavily edited to switch from plain text to icons. It definitely had an influence because it gave our app a graphical and easy-to-use look that worked for even the youngest kids.

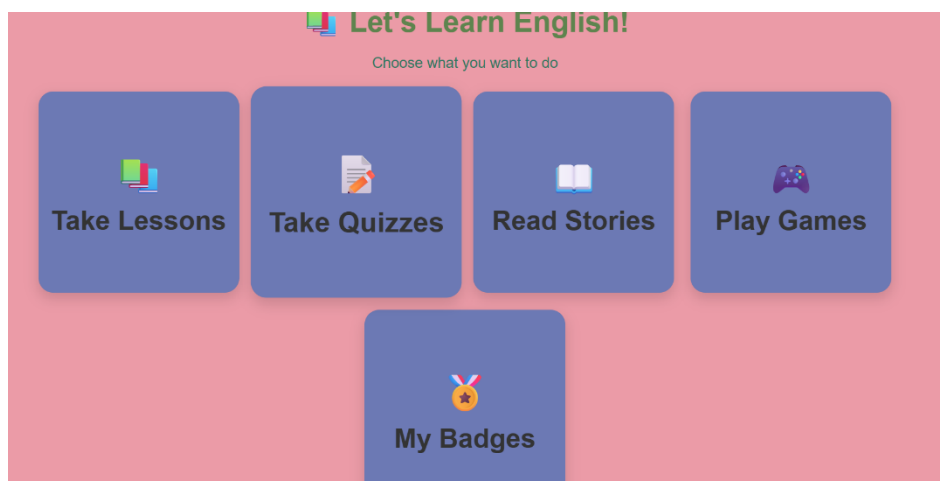


Figure 8: Main menu

Lesson Module

The lessons module stands as the backbone of the functionality of the platform. Instead of having set content for the lesson, dynamic loading of the lesson takes place, based on the language preference of the user. With respect to the lessons, five different types of lessons have been entirely constructed within the platform:

- **Colors Lesson:** This lesson helps children learn about colors by performing various activities. Random prompts, touch selection and real-time voice prompts are some of the ways in which the process of teaching is made engaging and interesting for the child.
- **Fruits Lesson:** This particular lesson aims at teaching children about various fruits and is done by taking up one fruit at a time. Pronunciation audio and activity-based lessons are also incorporated in this process.
- **Alphabet Lesson:** Letters will be laid out in a grid form whereby children can select each letter for pronunciation and example words starting with that particular letter will also be displayed.
- **Animals Lesson:** This lesson focuses on making children learn about various animals using images, animation and interactive activity modules spanning multiple rounds.
- **Shapes Lesson:** In this lesson, various shapes are taught to the child using interactive activities based on instructions received by voice prompts.

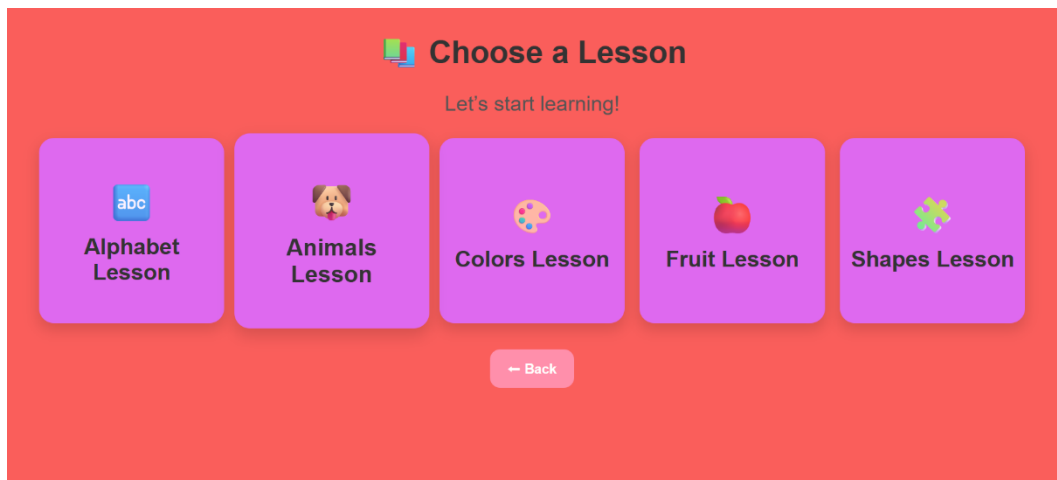


Figure 9: Lessons page

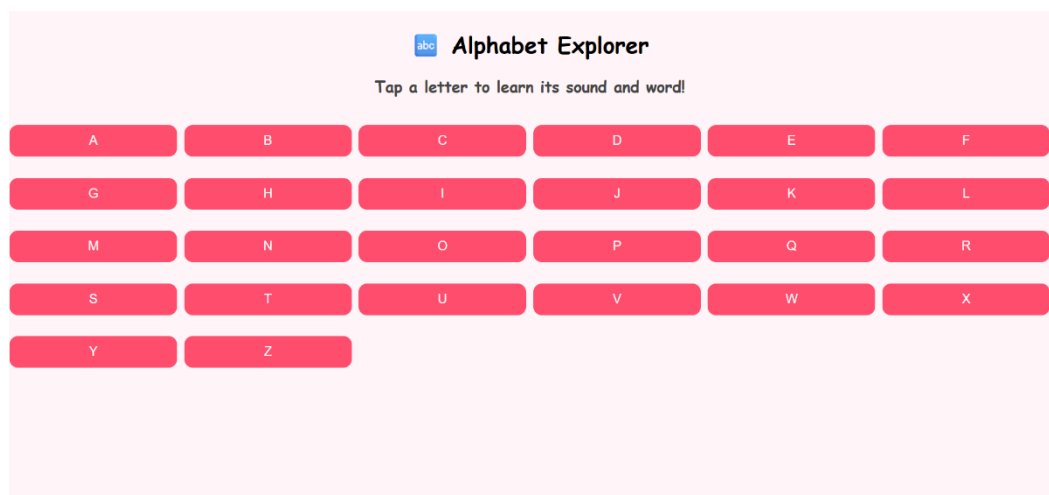


Figure 10: Example lessons interface

Quiz Module

The quiz section permits kids to test what they know by taking interactive quizzes involving not just words in the area but also their knowledge of specific units of learning in this area. By means of the same method as in the lessons, the quizzes get dynamically retrieved from the back-end depending on which category they have chosen. Children are allowed to choose five different categories on the quiz menu (colors, alphabet, shapes, animals and fruits), then they receive feedback on which answers are correct and incorrect during the quiz, in a way to motivate them in their learning process. Of all the quizzes, the one in the Shapes category is the most difficult to create and has elements including activities that are about shapes identification (both individually or in real-world items), finding odd-one-out between shapes etc.

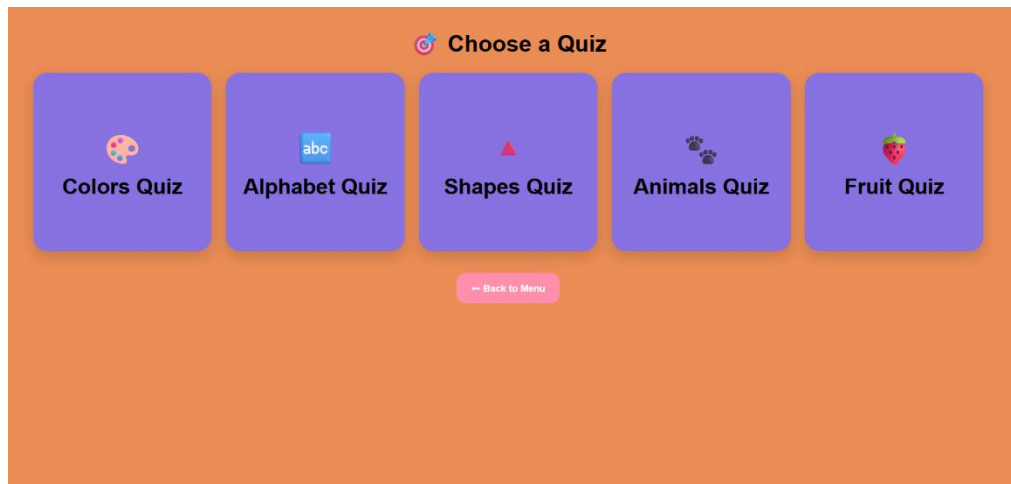


Figure 11: Quiz Menu

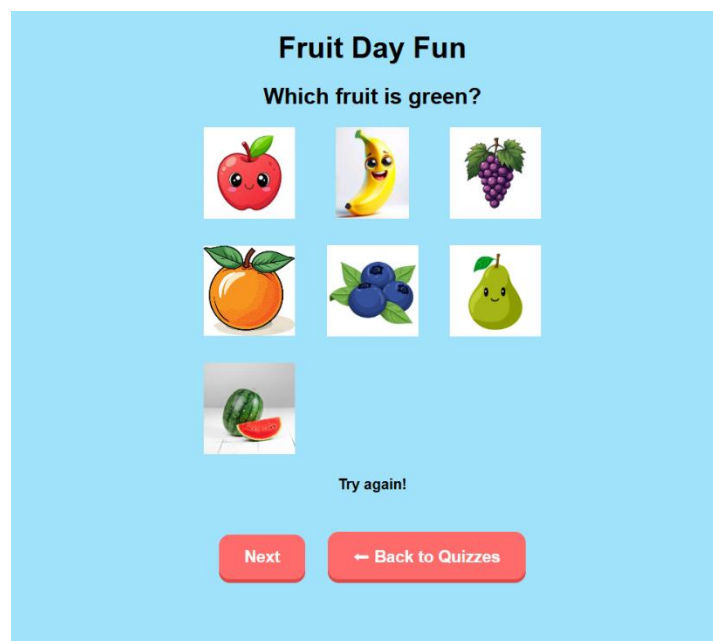


Figure 12: Quiz Interface Example

Stories Module

The stories section provides children with an entire library of interesting stories that will help them learn languages. The content of stories is dynamically generated from the server and presented in a visually appealing form in a card format with small emoji-style images to make the process more exciting and interesting for kids.

Choosing one of the stories will allow the child to move on to a new page that will enable him or her to read and listen to a narrator at the same time.

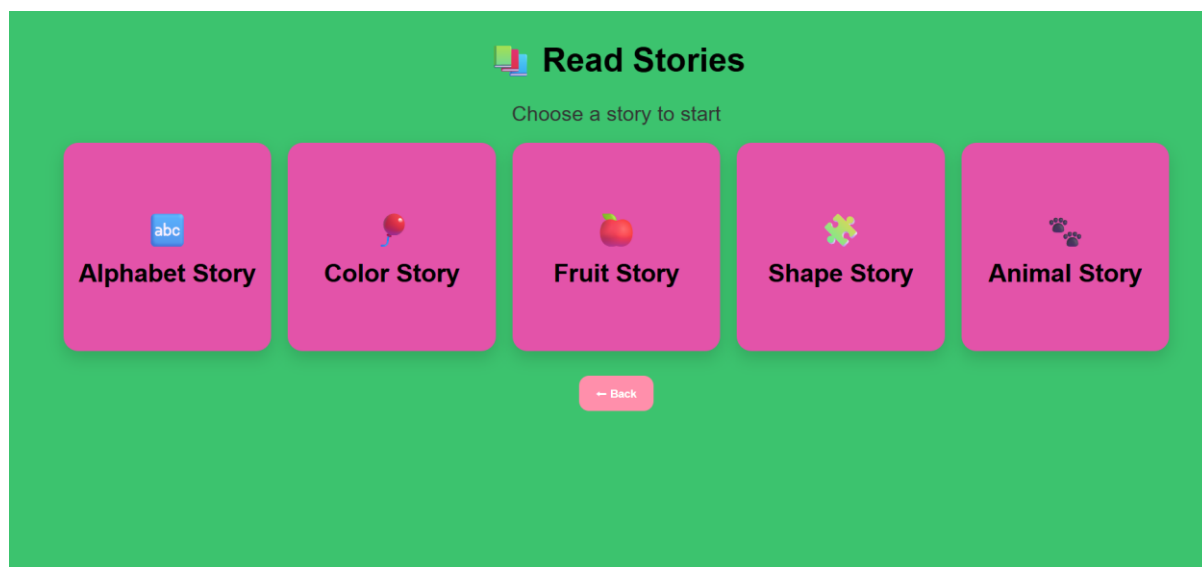


Figure 13: Stories page

Games Module

The games came together to present a lot more of an interactive and fun experience for the kids. Five games are already in existence, each devoted to a different word of language and some specific activity designed to encourage learning.

- **Magic Paint Splash (Colors):** Children select the colors to paint different objects, and correct answers give them cool bouncing and glowing animation effects.
- **Letter Builder (Alphabet):** Kids build simple words out of shuffled letters. These types of pictures are APPLE, BALL, CAT, DOG, FISH.
- **Animal Homes:** Kids use the pictures you give them and match the same animals with their habitats in this drag-and-drop matching game.
- **Shape Builder Town:** Kids use geometric shapes in the right places to build things — houses, rockets, trees, boats — out of their knowledge of geometry.
- **Guess the Fruit (Fruit Mystery):** Children will guess what fruit hides behind a string of clues and after successfully guessing the right fruit, the result will be revealed via an animated effect.

These different types of games help create a fun educational environment for the children.

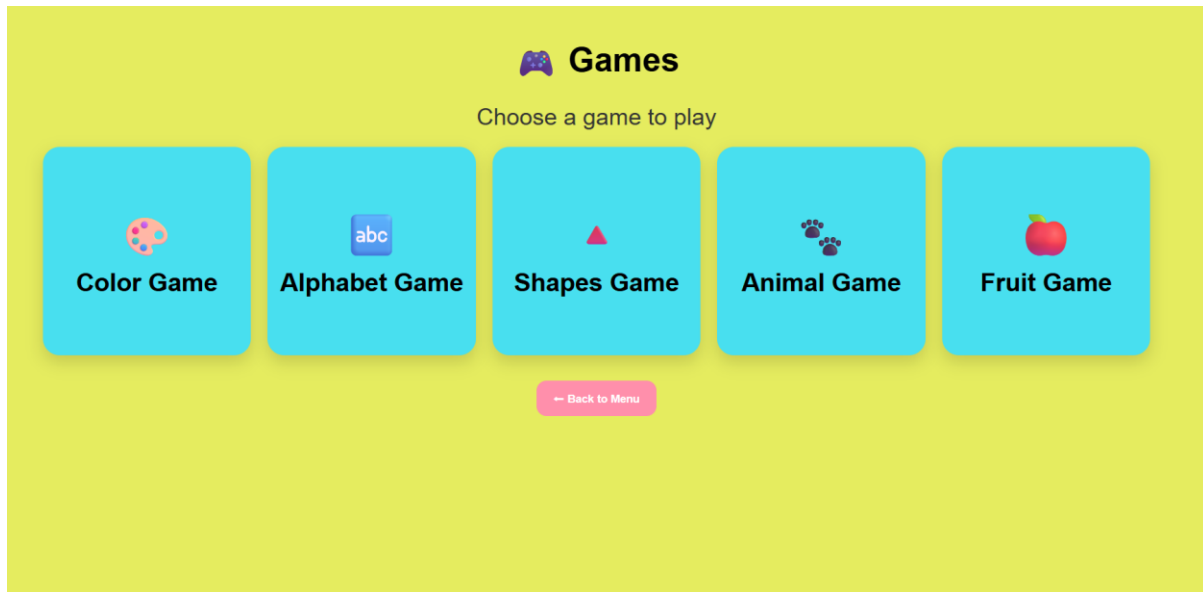


Figure 14: Games menu

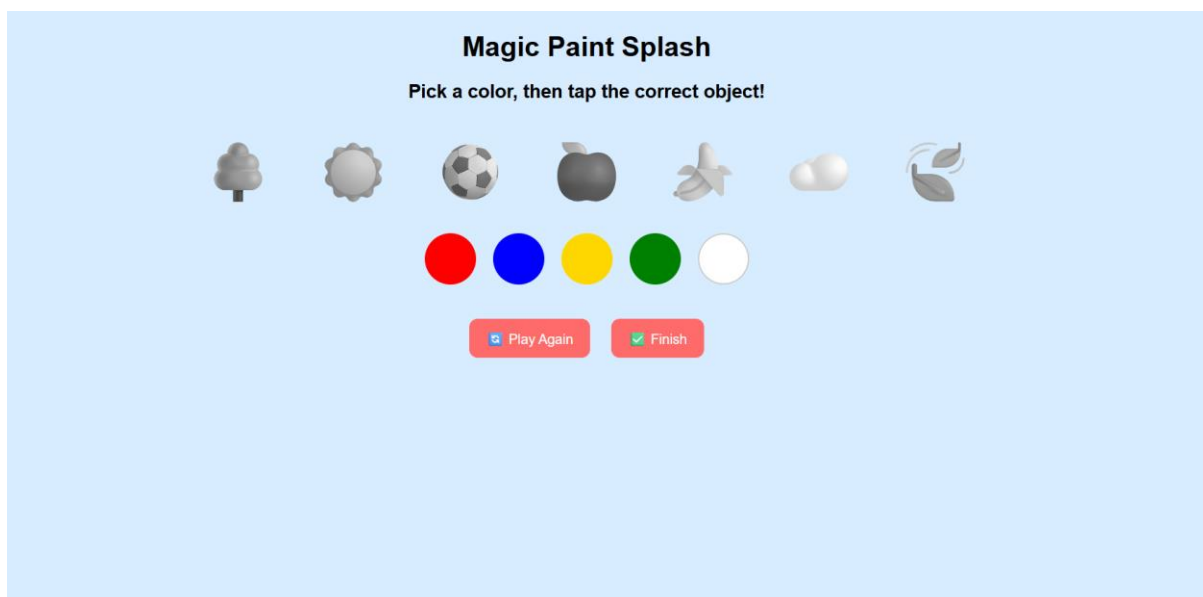


Figure 15: Example Game Interface

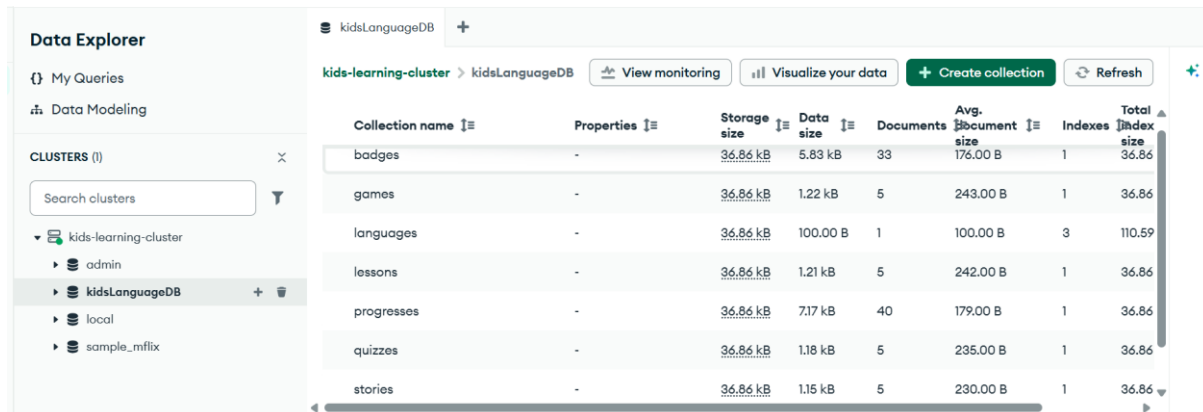
5.3 Backend Implementation

Overview

Child Language Learning System Backend The backend of Kids Language Learning System is built on Node.js and Express.js. Modularity in the system infrastructure allowed for the separation of model, route, and server setup files into more manageable and organized parts. It is essential to choose this approach to make sure that our backend can be scaled and controlled when new features are to be incorporated in the future.

Database Setup

The database is cloud-hosted which means a MongoDB on cloud based service such as MongoDB Atlas is hosted. The server will have access via the connection string located in environment variables to the database and maintain the confidentiality of the sensitive data and not push the credentials to the repository. The database is structured in collections of entities present in the application.



Collection name	Properties	Storage size	Data size	Documents	Avg. Document size	Indexes	Total index size
badges	-	36.86 kB	5.83 kB	33	176.00 B	1	36.86
games	-	36.86 kB	1.22 kB	5	243.00 B	1	36.86
languages	-	36.86 kB	100.00 B	1	100.00 B	3	110.59
lessons	-	36.86 kB	1.21 kB	5	242.00 B	1	36.86
progresses	-	36.86 kB	7.17 kB	40	179.00 B	1	36.86
quizzes	-	36.86 kB	1.18 kB	5	235.00 B	1	36.86
stories	-	36.86 kB	1.15 kB	5	230.00 B	1	36.86

Figure 16: MongoDB Atlas – Database Collections

Implemented Modules

Various modules have been created in the backend to implement various functionalities of the website:

- User Module:** It deals with all functionalities that deal with registration and login of users, involving usernames, email addresses, password, and role type.
- Language Module:** It deals with all the languages that can be chosen as the preferred language for learning. Every language has a unique ID that links its respective data.
- Lesson Module:** This module allows the user to upload new lessons and retrieve lessons in the chosen language.
- Story Module:** It enables stories in the language to be uploaded as well as retrieved from the database.
- Quiz Module:** The module is capable of dynamically loading quizzes associated with a particular lesson and language.
- Games Module:** The games module ensures that games are stored in the database so that they can be loaded dynamically.
- Progress Module:** User progress is stored using this module.

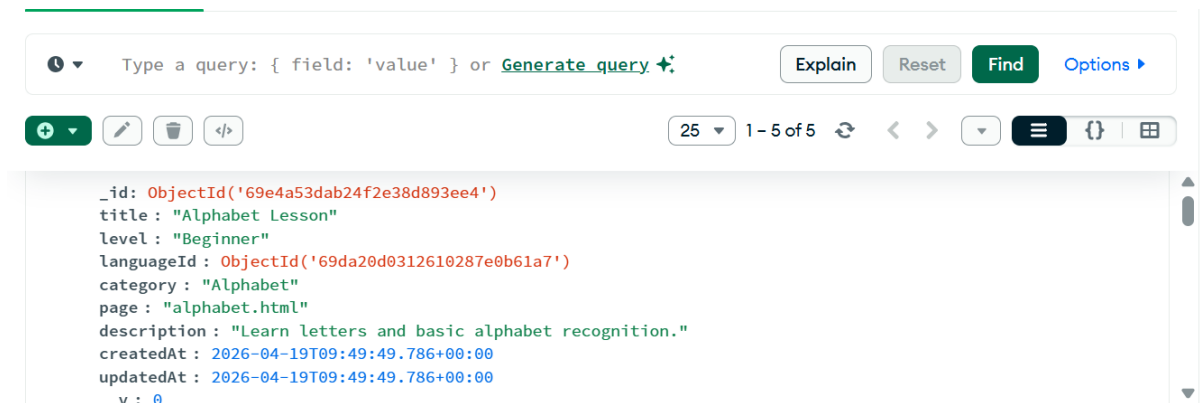


Figure 17: MongoDB Atlas – Sample Lesson Document

API Architecture

A RESTful API design was implemented to facilitate interactions between the front-end and back-end processes [20]. Prior to integrating the APIs into the front-end, all the endpoints had been checked for functionality using Thunder Client. The following API routes were implemented and tested within the system

API Route	Method	Function
/api/users/register	POST	Register a new user
/api/users/login	POST	Authenticate a user and return session data
/api/languages/add	POST	Add a new language
/api/languages	GET	Retrieve all available languages
/api/lessons/add	POST	Add a new lesson
/api/lessons	GET	Retrieve all lessons
/api/lessons/language/:languageId	GET	Retrieve lessons by language
/api/stories/add	POST	Add a new story
/api/stories	GET	Retrieve all stories
/api/quizzes/add	POST	Add a new quiz
/api/quizzes	GET	Retrieve all quizzes
/api/games/add	POST	Add a new game
/api/games	GET	Retrieve all games
/api/progress/add	POST	Save a progress record

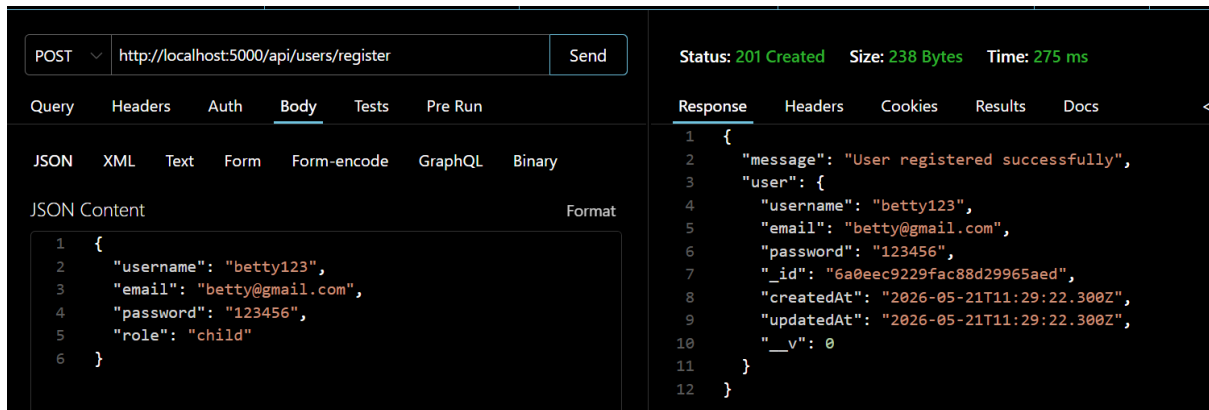


Figure 18: Thunder Client – API Testing

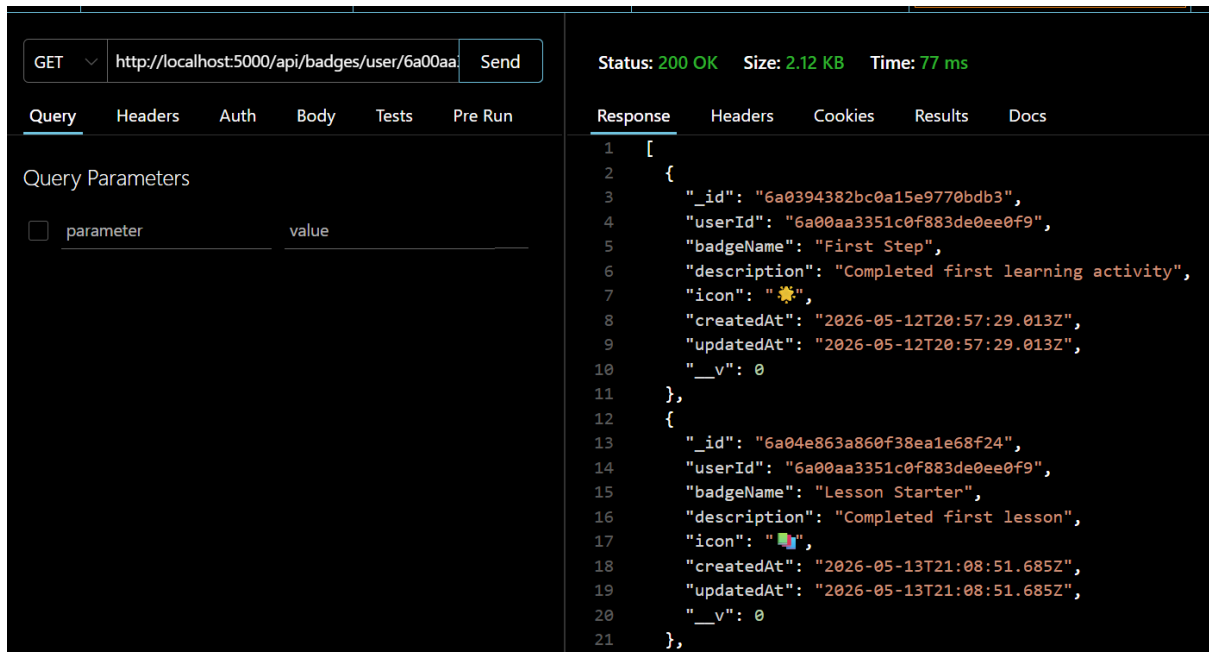


Figure 19: Example Badge System

5.4 Security Considerations

Considerations were made for security while building the backend application. The following security aspects were considered during development in order to make the application secure, along with recommendations for the next phase of the project and these security measures were recommended by OWASP security standards [22].

- Environmental Variables:** Private information like database strings and configuration values are stored in environmental variables. These files are ignored by .gitignore in order to keep them private.

•**Route Validation:** Validation of input values is done on the routing layer in order to ensure that there will be no bad data inserted into the database.

•**Authentication:** User's credentials are authenticated based on the record stored in the database.

• **Password hashing :** This will help to have passwords hashed with bcrypt [23]. This version is very simple but suitable for a prototype system. To fully scale, more security measures would be needed for the system, such as password hashing, JWT-based session management, and secure HTTPS configuration for better user and system access security[24].

TESTING AND VALIDATION

6.1 Testing Strategy

Testings were carried out all through the development process following standard software engineering testing methods [25], rather than waiting till the final stage of the development process. All the modules were tested as soon as they were created in order to spot and correct any errors as early as possible.

Various techniques of testings were employed such as browser-based manual testing, API testings, which utilized Thunder Client, and database testing, which utilized MongoDB Atlas Data Explorer. Four specific categories were examined during the testing procedure in order to guarantee proper functionality on both ends.

•**API Testing:** All backend APIs were tested separately using Thunder Client prior to their integration with the frontend part of the system. This was done to ensure smooth data creation, storage, and retrieval from the database.

•**Frontend Integration Testing:** After proper testing of the backend parts of the system, data display and fetching was performed on the frontend part of the project.

•**User Flow Testing:** All user flows were tested end-to-end including registration, login, language selection, learning lessons, answering quizzes, reading stories, and playing educational games.

6.2 Test Cases

Test Case	Description	Expected Result	Actual Result	Status
TC-01	User Registration	New user saved to database with correct fields	User record created in MongoDB	PASS
TC-02	User Login	Registered user authenticated and redirected to menu	Login successful, menu loaded	PASS
TC-03	Add Language	New language stored with unique ID	Language document created in DB	PASS
TC-04	Retrieve Languages	All languages returned from GET endpoint	Correct JSON array returned	PASS
TC-05	Add Lesson	Lesson linked to languageId and stored	Lesson document created in DB	PASS
TC-06	Retrieve Lessons by Language	Only lessons for selected language returned	Filtered results correct	PASS
TC-07	Dynamic Lesson Loading	Frontend displays lessons from API, not hardcoded	Lesson cards rendered from DB data	PASS

TC-08	Quiz Score Calculation	Score calculated correctly at end of quiz	Score displays accurately	PASS
TC-09	Story Dynamic Loading	Stories loaded from backend and displayed as cards	Story cards rendered from DB data	PASS
TC-10	Game – Letter Builder	Correct word assembled from tiles triggers success	Completion animation triggered correctly	PASS
TC-11	Progress Data Saved	Completed lesson sends progress record to backend	Progress document saved in DB	PASS
TC-12	Invalid Login	Wrong credentials rejected with error message	Error message displayed, no redirect	PASS
TC-13	Missing Route Export	Express router correctly exports handler functions	Routes respond correctly	PASS (after fix)
TC-14	MongoDB Connection	Backend connects to Atlas using environment variable	Connection established on server start	PASS

6.3 Test Results and Evaluation

After fixing the problems that were found, all fourteen test cases ran successfully. The testing problems mainly fell into three categories:

Route Configuration Errors

Some Express.js routes were showing errors like “argument handler must be a function.” This happened because the router files had wrong exports, such as missing `module.exports` or exporting the wrong object. These issues were fixed by carefully checking all the route files and making the export style the same in every file.

MongoDB Integration Issues

At first, the database would not connect because the connection string was wrong, some environment variables were missing, and some model file paths were incorrect. These problems were fixed by checking the imports, correcting the connection string, and making sure the `.env` file was working properly.

Frontend-Backend Communication

Many fetch API requests failed in the beginning because of missing `http://` in URLs, incorrect frontend-to-backend routes, and wrong field names in request bodies. These problems were found and fixed by using browser console logs and testing requests with Thunder Client.

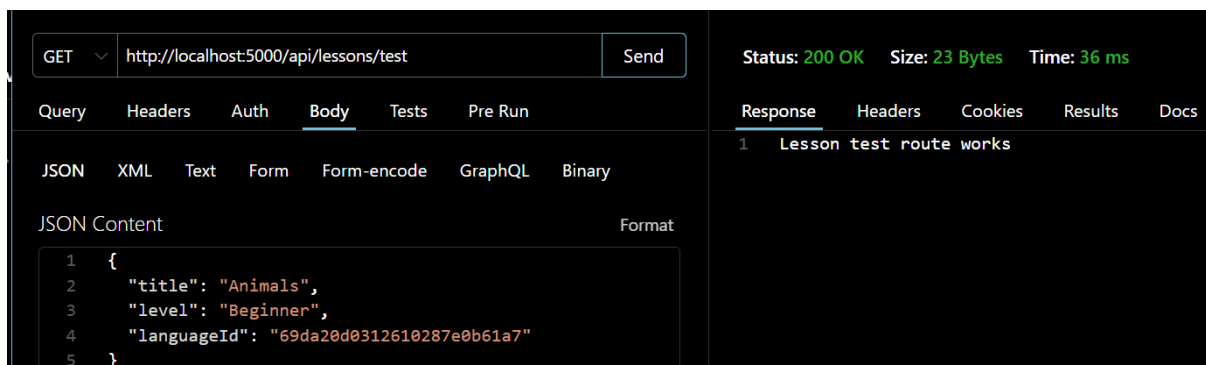


Figure 20: API Testing – Lesson Retrieval by Language

RESULT AND DISCUSSION

The following table summarizes the implementation status of all planned features at the time of project submission:

Feature	Status	Notes
User Registration	Complete	Fully functional with MongoDB storage
User Login	Complete	Verified against database credentials
Language Selection	Complete	English fully implemented; others architecture-ready
Lessons Module	Complete	5 categories: Colours, Alphabet, Fruits, Animals, Shapes
Stories Module	Complete	Dynamic loading from backend with voice narration
Quiz Module	Complete	5 quiz categories with score calculation
Games Module	Complete	5 games across all lesson categories
Backend API Integration	Complete	14 REST API endpoints fully tested
Dynamic Frontend Rendering	Complete	All content loaded from database, not hardcoded
MongoDB Database Integration	Complete	MongoDB Atlas with 7 collections
Voice Guidance (Speech Synthesis)	Complete	Integrated across lessons and stories
Progress Tracking	In Progress	Data model and API endpoint complete; UI display pending
Badge and Reward System	Planned	Database model designed; award logic pending
Admin Dashboard (UI)	Planned	Currently managed through API endpoints only
Mobile Optimisation	Partial	Responsive design implemented; further testing needed

The main goals of the project were achieved by creating a full-stack interactive language learning platform for children. The project started as a simple HTML frontend, but later became a dynamic application with a database, user login system, and many educational features.. For two developers making contributions within one academic semester, this is a powerful technical feat, involving frontend and backend development as well as system integration. My main technical learning experience from this project was working on backend integration. Shifting from some kind of backend system that relies on hardcoded content to one that pulls from a real database on the fly meant reworking how everything moves through the application.

This consisted of how in the event of a query the frontend requests data, how asynchronous response is handled and how transient state is addressed between pages, by using the localStorage. The implementation of all of these components took its time, and stabilizing the

resulting final version was one of the most difficult parts of this project. The games module is a bit more difficult on the frontend side as well. Making interactive games in vanilla JavaScript meant state manipulation, animations, timers, and running each of the game's various gameplay codes without the need for a framework, this requirement resulted in much laborious pre-design and post-testing. Even though it's pretty advanced, the last five games are working fine and run smoothly, offering an engaging and enjoyable experience for users

LIMITATIONS OF THE SYSTEM

Although the Kids Language Learning System functions as a prototype successfully, it has limitations.

Password Security: Currently passwords are unencrypted and these are unsafe. The prototype is working well while it is tested, however, bcrypt-type security means the system will not be accessible to real users until using it is strengthened.

Administrative Dashboard: There is no admin dashboard yet, so administrators cannot handle content with a simple interface. Content will only be controlled with API calls using tools such as Thunder Client.

Progress Tracking: The backend can track user progress, but the frontend does not yet offer completion of lessons or performance summaries to users.

Badge System: A badge system and a database model have been developed but nothing has been implemented for giving badges and showing them to users.

Single Language Support: The app should have many languages; it currently supports only English. Additional languages are upcoming but not completed.

Lack of Offline Features: It requires an Internet connection for working, and lacks any offline support at this time for caching or storage.

Mobile Optimization: The system is available on various devices but on desktops and tablets it has been most tested. It might require some more improvement in cases of small phone screens. * **Browser Compatibility:** The voice features rely on web browser speech synthesis and aren't going to work in the same way on every browser.

FUTURE WORK AND ENHANCEMENTS

The Kids Language Learning System is built for a coming generation with improvements in mind, which leads to how much more it could be improved in the future.

Advanced Progress Tracking

Though the system saves progress data, it could communicate the info in a clearer and more useful way. A progress dashboard could reflect finished lessons, quiz scores and the areas in which a child is successful or needs more practice. This allows children, parents and teachers to see learning progress over time easily.

Admin and Teacher Dashboard

If we added a visual admin dashboard, it would make the system easier to use—especially for teachers and administrators who don't know anything about coding. They could manage lessons and content via a simple interface instead of API endpoints. An alternative teacher dashboard would also make the performance and learning progress of students more transparent.

Password Encryption and Security

Stronger features must be implemented for user application prior to system usage for real users. User passwords should be encrypted with bcrypt hashing. JWT authentication could be used instead of the existing localStorage for secure login sessions. HTTPS should also be employed to maintain security of the users' personal information while communicating.

Multilingual Content Expansion

Multiple languages will be supported with the system to encourage multilingual learning benefits in children [27], but only English language is supported till now. More and more languages, such as Spanish, French, Chinese and Swahili, could be added in the future to educate more children in multiple languages.

Adaptive Difficulty

The system was designed to accommodate varying degrees of difficulty. Quizzes and lessons could be made easier or more difficult in the future, depending on how well a child performs. This would promote an additional personalized learning experience through some adaptive learning approaches [26].

Deployment

The plan for the project is open source deployable online. The Node.js backend might be hosted on Render or Railway, the frontend on Vercel. We already use MongoDB Atlas for

database. The system could be deployed online, so real users could interact with the tool and test it live.

User Testing with Children

Testing the system with real children is still needed. Observing the way the kids interact with the platform would reveal any confusing aspects, learn what features they find most effective to use, and make a better overall user experience overall.

CONCLUSION

The Kids Language Learning System was made as a full-stack web app with interactive features like lessons, quizzes, stories, and games to help kids learn language in a fun way. The goal was achieved successfully. It started as simple static HTML pages, but it later became a working app that uses a database and has much more functionality. Over time, user authentication, dynamic content loading, RESTful API integration, voice guidance, and other interactive educational games were included in this system. So, the main success of the project was going from a static system with hardcoded content to a dynamic system, where the actual data is on a real database and served using well-tested API endpoints. That was a success. From a couple of basic static HTML pages, we've now built a fully functional, database driven application. Subsequently user authentication, dynamic content loading, RESTful API integration, voice guidance, and many other interactive educational games have been added. So the real success of the project was going from a static system (hardcoded content) to something that's a real platform where data gets stored in a real database and where well-tested API endpoints are serving it.

The Kids Language Learning System was created in order to develop an interactive full stack web application to teach kids language concepts through lessons, quizzes, stories and games. That was a success. What started as a few simple static HTML pages, is now a fully functioning database driven application. Over the years, user authentication, dynamic content loading, RESTful API integration, voice guidance, and a variety of interactive educational games have also been added. The key achievement of this project was to transition it from a simple static system with hard-coded content to a more powerful application for a real database being used that works through tested API endpoints. The Kids Language Learning System is a full-stack web app to assist kids in learning language through lessons, quizzes, stories, and games. This particular aim has been attained. What began as a few simple static HTML pages has grown all the way to a fully working application powered by a database. Eventually, elements like user sign-in, dynamic content load, API links, voice support, and interactive learning games were incorporated.

The project's greatest success was the transition from a static system with hard coded content to a dynamic platform, with the data being stored in a real database and served through well tested API endpoints.

The Kids Language Learning System was developed as an interactive full stack web application to allow children to learn language concepts through lessons, quizzes, stories and games. This was successful.

A few simple static HTML pages was the beginning of the project, which progressed into a fully working database driven application later on. Things like user authentication, dynamic content loading, integration with RESTful APIs, voice guidance, and various interactive educational games have been integrated throughout the years. Most of the success of the project has come from making this transition from a static framework with hard coded content to a dynamic base, stored in a real database and served through well-tested API endpoints.

REFERENCES

- [1] A. Holmes and K. Tuomi, “State of the art and practice in AI in education,” *European Journal of Education*, vol. 57, no. 4, pp. 542–570, 2022.
- [2] UNESCO, “Education during COVID-19 and beyond,” *UNESCO Reports*, Paris, France, 2020.
- [3] R. Pinter, “Gamification in Education: Does it Improve Learning Outcomes?,” *Journal of Educational Technology*, vol. 18, no. 2, pp. 45–62, 2021.
- [4] K. Kapp, *The Gamification of Learning and Instruction*. San Francisco, CA, USA: Pfeiffer, 2012.
- [5] J. Piaget, *The Psychology of the Child*. New York, NY, USA: Basic Books, 1969.
- [6] L. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA, USA: Harvard University Press, 1978.
- [7] M. Prensky, “Digital Natives, Digital Immigrants,” *On the Horizon*, vol. 9, no. 5, pp. 1–6, 2001.
- [8] Duolingo ABC, “Learn to Read with Duolingo ABC,” 2025. [Online]. Available: <https://abc.duolingo.com/>
- [9] Khan Academy Kids, “Learning App for Kids,” 2025. [Online]. Available: <https://learn.khanacademy.org/khan-academy-kids/>
- [10] Lingokids, “Playlearning App for Kids,” 2025. [Online]. Available: <https://lingokids.com/>
- [11] Starfall Education Foundation, “Starfall Education,” 2025. [Online]. Available: <https://www.starfall.com/>
- [12] M. Ally and A. Tsinakos, *Increasing Access through Mobile Learning*. Vancouver, Canada: Commonwealth of Learning, 2014.
- [13] WHATWG, “HTML Living Standard,” 2025. [Online]. Available: <https://html.spec.whatwg.org/>
- [14] MDN Web Docs, “JavaScript Guide,” 2025. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>
- [15] Node.js Foundation, “Node.js Documentation,” 2025. [Online]. Available: <https://nodejs.org/en/docs>
- [16] Express.js, “Express Web Framework,” 2025. [Online]. Available: <https://expressjs.com/>
- [17] MongoDB Inc., “MongoDB Atlas Documentation,” 2025. [Online]. Available: <https://www.mongodb.com/docs/atlas/>
- [18] MDN Web Docs, “Web Speech API,” 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
- [19] D. Norman, *The Design of Everyday Things*. New York, NY, USA: Basic Books, 2013.

- [20] R. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, USA, 2000.
- [21] Mongoose, “Mongoose Documentation,” 2025. [Online]. Available: <https://mongoosejs.com/docs/>
- [22] OWASP Foundation, “OWASP Top Ten Web Application Security Risks,” 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [23] N. Provos and D. Mazieres, “A Future-Adaptable Password Scheme,” in Proceedings of the USENIX Annual Technical Conference, 1999.
- [24] JWT.io, “Introduction to JSON Web Tokens,” 2025. [Online]. Available: <https://jwt.io/introduction>
- [25] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2015.
- [26] J. Brusilovsky and E. Millán, “User Models for Adaptive Hypermedia and Adaptive Educational Systems,” in The Adaptive Web. Berlin, Germany: Springer, 2007, pp. 3–53.
- [27] E. Bialystok, Bilingualism in Development: Language, Literacy, and Cognition. Cambridge, UK: Cambridge University Press, 2001.